GPU Computing
Exercise 2

Luis Wenzel
Miro Joensuu
Daniel Bayer
Noah Schneider

1 Raw Kernel Startup Time

1.1 Raw Kernel Startup Time

The first messurement already shows, that a simple usage of asynchronous versus synchronous execution can improve the runtime of code by 2,92 micro seconds.

```
 chTimerGetTime( &start );
 for ( int i = 0; i < cIterations; i++ ) {
        NullKernel<<<1,1>>>();
}
cudaDeviceSynchronize();
chTimerGetTime( &stop );
```

Listing 1: Asynchrounous NullKernel start with 1 Grid 1 Block

```
chTimerGetTime( & start_syn );
        for ( int i = 0; i < cIterations; i++) {
                NullKernel<<<1,1>>>();
                cudaDeviceSynchronize();
        }

        chTimerGetTime( &stop_syn );
```

Listing 2: Synchrounous NullKernel start with 1 Grid 1 Block

The Execution of the Synchronization is here by put outside of the loop. The next Task explores the usage of Grids and Blocks in a GPU. Herefore the NullKernel Task of the Previous execution is used. By varying the grid size or the block size for synchrounous or asynchrounous tasks, it is visible that in both options the asynchronious task are faster then the synchronious. The grafik 1 shows, that by in smaller grid sizes, both kernels maintain a nearly constant runtime. However, when the number of blocks increase beyond

several thousand, the runtime rises sharply. This sugests, that larger grids increase the scheduling and management overhead on the GPU.

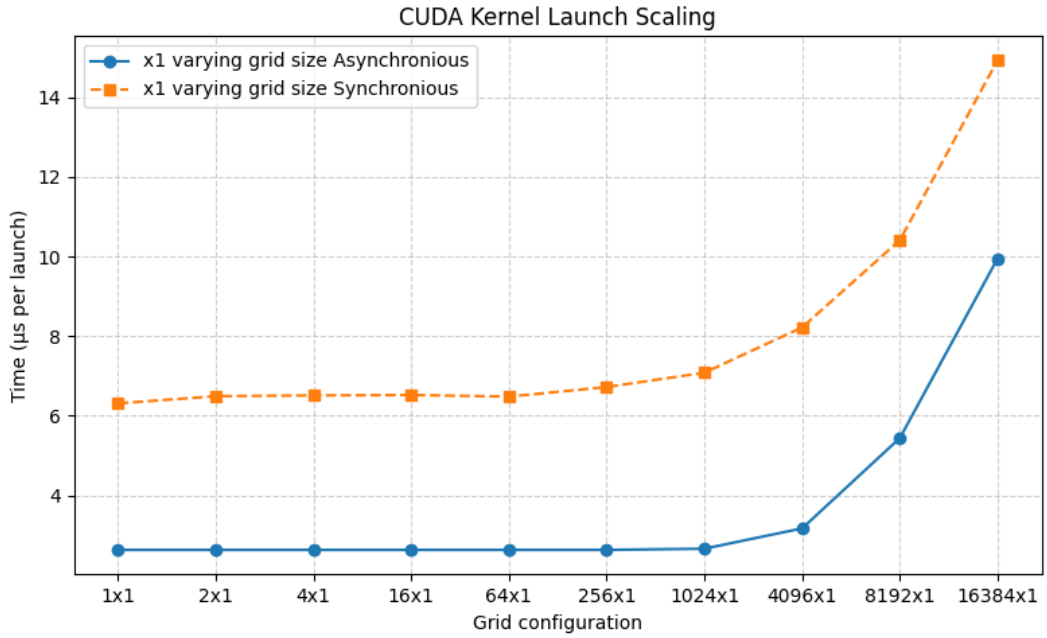

Figure 1: exercise 2.1, asynchronous vs. synchronous varying grid single block

The second messurement of the NullKernel execution time, where the block size changes, shows very little runtime difference for eighter Asynchronious nor Synchronious execution (this shows the graf 2). Since the Asynchronious execution of Tasks is in both Cases is the faster mode, it is underlinned, that only the overall amount of blocks has an influence onto the runtime. This comes frome the fact that only there the overall amount to administrate these blocks and scedule those is tasks is increased. All in all the overhead scales in grid size, but not with block size. And the Asynchronious execution reduces the kernel dispatch overhead significantly.

2 Break-even Kernel Startup Time

2.1 Break-even Kernel Startup Time

We can see that the asynchronous kernel startup time stays roughly constant until a processing time of around 3100 clock cycles, after which it starts rising linearly with increasing clock cycles. This indicates that until 3100 cycles the

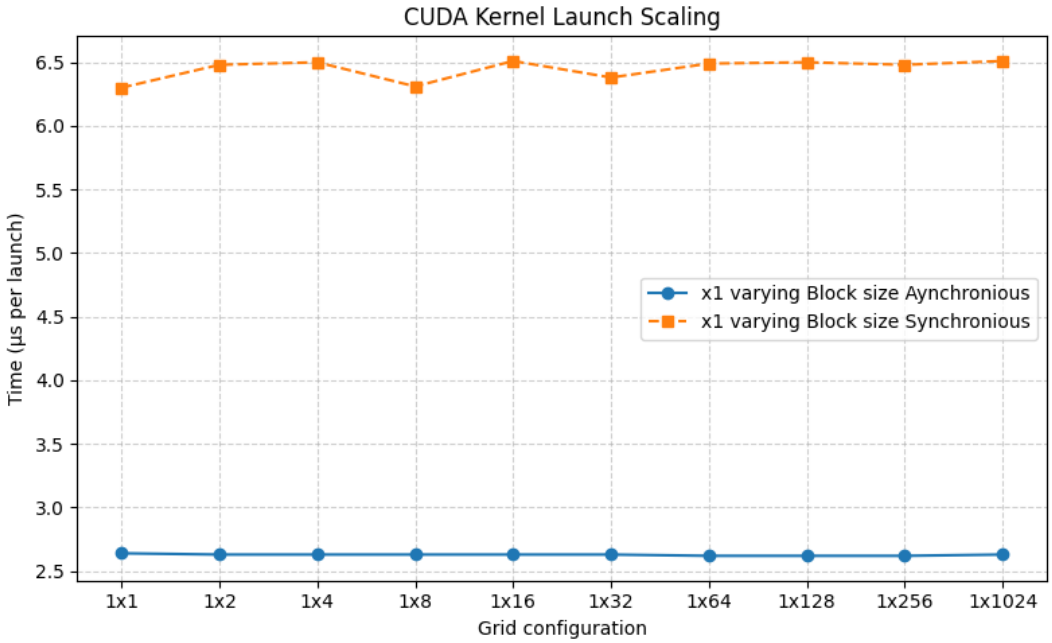

Figure 2: exercise 2.1, asynchronous vs. synchronous single grid varying block

GPU spends time idle even after the kernel is finished processing. After 3100 cycles, the next kernel can be directly launched without additional overhead. Roughly 8200 clock cycles are needed until the initial asynchronous time of around 2.7 s doubles. This time of around 5.3 s consists of 2.7 s for the kernel startup time in addition to another 2.7 s coming from time only spent processing. In this time (5.3 s or 8200 clock cycles), another kernel launch call could have been completed.

3 PCIe Data Movements

3.1 PCIe Data Movements

We see that throughput increases for both pinned and pageable memory with increasing data size. This is because of data overhead that dominates at small data sizes. As the message size increases the overhead becomes less relevant, and we approach peak bandwidth. In both cases (D2H and H2D) pinned memory outperforms pageable memory, but especially so in the D2H case. This is because of the GPU overhead and necessary staging of the paged data, when accessing it from the host. Using pinned memory, we get

a peak bandwidth of 13.1 GB/s for D2H (5.7 GB/s using pageable memory), close to the peak PCIe connection of 15.8 GB/s. For the host, we get a peak bandwidth of 12.5 GB/s using pinned memory and 11.9 GB/s using pageable memory.

4 Presenting

Willing to present:
2.1: Yes
2.2:
2.3: